Dev Guide - Teleprompter Text Positioning (Recording Studio)

Goal of this change: keep the live camera fullscreen in every case, and move only the teleprompter text depending on the device:

Situation	Camera	Teleprompter text
Laptop / desktop (wide, non-touch)	Fullscreen	Top, horizontally centered
Touch device - portrait / tall	Fullscreen	Top, horizontally centered
Touch device - landscape / sideways	Fullscreen	Far left, vertically centered (top->bottom)

A laptop and a sideways tablet are *both* "landscape," so screen orientation alone can't tell them apart. We detect touch capability (coarse pointer) to distinguish them.

Scope

- One file changed: artifacts/client/src/studio/RecordingStudio.tsx
- No new dependencies, no API changes, no DB changes, no server changes.
- Client-only CSS/JSX. Safe to ship with a normal frontend deploy.
- The <video> (camera) element is not touched - it stays fullscreen.

Change 1 - Add a touch-device flag

Add this state right after the existing viewportPortrait state declaration (near the top of the component, ~line 190):

```
// Whether this is a touch device (coarse pointer). Used to tell a tablet held
// sideways (landscape, touch) apart from a laptop/desktop (also "landscape"):
// the sideways tablet puts the teleprompter on the far left, the laptop keeps
// it top-centered. Capability is effectively static, so compute once.
const [isTouch] = useState<boolean>(() => {
  if (typeof window === "undefined") return false;
  const mm = window.matchMedia;
  const coarse =
    (mm?.("(pointer: coarse)").matches ?? false) ||
    (mm?.("(any-pointer: coarse)").matches ?? false);
  const hasTouchPoints =
    typeof navigator !== "undefined" && navigator.maxTouchPoints > 0;
  return coarse || hasTouchPoints;
});
```

> *Note: this uses OR logic (coarse pointer *or* maxTouchPoints) so a touch device with a stylus/trackpad (which reports a fine primary pointer) still classifies as touch.*

Change 2 - Derive the placement flag

Right after the existing const portrait = viewportPortrait; line (~line 696):

```
// Teleprompter placement: top-centered everywhere (portrait, and laptop/desktop)
// EXCEPT a touch device held sideways (landscape + touch), where the text moves
// to the far left and centers top-to-bottom. Camera stays fullscreen in all cases.
const textLeftCentered = !portrait && isTouch;
```

Change 3 - The teleprompter container styles

Find the teleprompter wrapper <div ref={prompterRef} style={{ ... }}>

(it has the comment {/* Teleprompter - large shadowed text directly over the video */}).

Before:

```
style={{
  position: "absolute",
  top: portrait ? "3.3rem" : "1.4rem",
  left: portrait ? 0 : "1.5rem",
  right: portrait ? 0 : "auto",
  width: portrait ? "auto" : ${promptWidth}%,
  maxWidth: portrait ? "100%" : "62%",
  height: ${prompterHeight}px,
  maxHeight: portrait ? "50%" : "72%",
  overflow: "hidden",
  padding: portrait ? "0 1.1rem" : 0,
  transform: mirrorText ? "scaleX(-1)" : "none",
  pointerEvents: "none",
  zIndex: 8,
}}
```

After:

```
style={{
  position: "absolute",
  height: ${prompterHeight}px,
  maxHeight: portrait ? "50%" : "72%",
  overflow: "hidden",
  pointerEvents: "none",
  zIndex: 8,
  // Three placements (camera stays fullscreen in all):
  //  - sideways touch device -> far left, vertically centered
  //  - portrait -> top, full width, centered text
  //  - laptop/desktop -> top, promptWidth-wide, horizontally centered
  ...(textLeftCentered
    ? {
        top: "50%",
        left: "1.5rem",
        right: "auto",
        width: ${promptWidth}%,
        maxWidth: "62%",
        padding: 0,
        transform: translateY(-50%)${mirrorText ? " scaleX(-1)" : ""},
      }
    : portrait
      ? {
          top: "3.3rem",
          left: 0,
          right: 0,
          width: "auto",
          maxWidth: "100%",
          padding: "0 1.1rem",
          transform: mirrorText ? "scaleX(-1)" : "none",
        }
      : {
          top: "3.3rem",
          left: "50%",
          right: "auto",
          width: ${promptWidth}%,
          maxWidth: "100%",
          padding: 0,
          transform: translateX(-50%)${mirrorText ? " scaleX(-1)" : ""},
        }),
}}
```

What each branch does

- textLeftCentered (sideways touch): anchored left: 1.5rem, vertically centered via top: 50% + translateY(-50%).
- portrait: pinned to the top (top: 3.3rem), full width, text centered.
- else (laptop/desktop): pinned to the top, promptWidth-wide, horizontally centered via left: 50% + translateX(-50%).

> *The mirrorText flip (scaleX(-1)) is composed onto the same transform string so the existing "Mirror text" toggle keeps working in all three modes.*

Change 4 - Inner text alignment

In the inner <div ref={prompterInnerRef} style={{ ... }}>, change the single textAlign line:

Before:

```
textAlign: "left",
```

After:

```
textAlign: textLeftCentered ? "left" : "center",
```

Verify after migrating

1. pnpm --filter @workspace/client run typecheck (or your repo's typecheck).
2. Manual smoke test of the studio:
   - Laptop browser: text top-center; camera fullscreen.
   - Phone/tablet portrait: text top-center; camera fullscreen.
   - Tablet/phone landscape: text far-left, vertically centered; camera fullscreen.
   - Toggle "Mirror text" in each mode - text should flip but stay positioned.
   - The Reading width slider should still resize text on laptop + sideways.
   - Start a recording and confirm the scroll animation still runs smoothly.

That's the complete change set - one component, four small edits.